

一、设计（论文）的基本要求（任务）

1.1 课题背景与意义

随着人工智能技术的快速发展，多智能体系统（Multi-Agent System）在医疗健康领域展现出巨大潜力。传统的单一智能体系统在处理复杂医疗问题时存在局限性，而多智能体协同系统能够模拟真实医疗场景中多学科团队协作的决策过程，提供更准确、更可靠的医疗咨询和辅助诊断服务。

本课题旨在构建一个基于多智能体协同的医疗健康咨询与辅助诊断系统，通过多个专业化智能体的协作，为患者提供初步健康咨询服务，并为医生提供智能化的辅助诊断支持。该系统具有重要的理论价值和实际应用意义：

理论价值：

- 探索自适应多智能体协作架构在医疗领域的应用
- 研究医学复杂度评估与智能体协作模式选择机制
- 探索强化学习在医疗智能体优化中的应用

实际应用价值：

- 提升医疗服务的效率和质量，减少医生重复性工作
- 为患者提供便捷的健康咨询渠道，改善就医体验
- 通过AI辅助，提升基层医疗服务质量，降低医疗门槛

1.2 设计目标

本毕业设计需要完成以下目标：

1.2.1 系统功能目标

患者端功能：

- 智能咨询对话**：实现多轮对话式健康咨询，支持文字输入，能够理解患者症状描述并给出初步建议
- 病历/报告上传**：支持图片、PDF格式的检查报告上传，实现OCR自动识别和数据提取
- 用药提醒管理**：提供用药时间、频率设置功能，支持定时提醒通知
- 个人中心**：实现历史咨询记录查询、病历摘要查看、个人信息管理等功能

医生端功能：

- 患者会话管理**：实现患者列表查看、会话详情展示、优先级排序等功能
- 智能辅助诊断界面**：展示患者主诉、智能体生成的结构化病历草稿（可编辑）、检查数据可视化、辅助诊断建议（诊断、鉴别诊断、建议检查）
- 知识库查询**：内嵌医学知识库检索，支持关键词搜索、分类浏览
- 病历归档**：实现病历确认、编辑、归档、导出、打印等功能

1.2.2 技术实现目标

- 多智能体协同系统**：
 - 实现基于MDAgents框架的自适应多智能体协作架构
 - 实现调节器代理（Moderator Agent），能够评估医疗查询复杂度并选择最优协作模式

- 实现PCC（Primary Care Clinician）模式：单个智能体处理低复杂度查询
- 实现MDT（Multidisciplinary Team）模式：3-5个智能体通过迭代讨论达成共识处理中复杂度查询
- 实现ICT（Integrated Care Team）模式：5-7个智能体分阶段协作处理高复杂度查询
- 智能体优化机制：
 - 实现多维度奖励评估系统（诊断准确性、信息获取效率、协作效率、遵循规范）
 - 实现两阶段训练优化机制（监督微调SFT + 强化学习RL）
 - 实现患者模拟智能体，为智能体提供训练和测试环境
- 病历结构化引擎：
 - 实现从非结构化患者主诉到结构化病历的自动转化
 - 使用NLP技术（NER、关系抽取）提取关键医疗信息
 - 生成符合医学规范的结构化病历模板
- 系统架构：
 - 采用前后端分离架构，后端使用FastAPI框架，前端患者端使用小程序，医生端使用Vue.js
 - 实现RESTful API接口设计，支持WebSocket实时通信
 - 实现用户认证与权限管理（JWT、RBAC）
 - 实现数据库设计（MySQL）和缓存机制（Redis）

1.3 设计要求

1.3.1 功能设计要求

- 完整性：系统应覆盖从患者咨询到医生诊断的完整业务流程，功能模块齐全
- 可用性：系统界面友好，操作简便，符合用户使用习惯
- 可靠性：系统应具备良好的错误处理机制，保证系统稳定运行
- 安全性：实现用户认证、权限控制、数据加密等安全机制

1.3.2 技术设计要求

- 架构设计：
 - 采用分层架构设计（表示层、业务逻辑层、数据层）
 - 实现模块化设计，各模块职责清晰，耦合度低
 - 支持系统扩展和维护
- 代码质量：
 - 代码结构清晰，注释完整
 - 遵循编程规范，使用类型提示
 - 关键模块编写单元测试，测试覆盖率不低于70%
- 性能要求：
 - API响应时间（P95）不超过500ms
 - 支持并发用户数不少于100
 - 智能体响应时间根据复杂度不同：PCC模式<15秒，MDT模式<100秒，ICT模式<230秒
- 文档要求：

- 编写完整的系统设计文档
- 编写API接口文档
- 编写用户使用手册
- 编写部署文档

1.3.3 创新性要求

- 技术创新：
 - 实现自适应多智能体协作机制，根据医疗查询复杂度动态选择协作模式
 - 实现多维度奖励评估和强化学习优化机制
 - 实现患者模拟智能体训练环境
- 应用创新：
 - 将多智能体系统应用于医疗健康咨询领域
 - 实现从非结构化主诉到结构化病历的自动转化
 - 提供可解释的AI诊断建议

1.4 论文撰写要求

- 论文结构：
 - 绪论：介绍研究背景、意义、国内外研究现状、研究内容
 - 需求分析：分析系统功能需求、非功能需求、用户需求
 - 系统设计：详细描述系统架构设计、功能模块设计、数据库设计、接口设计
 - 系统实现：描述关键技术实现、核心模块实现、系统界面展示
 - 系统测试：描述测试环境、测试用例、测试结果分析
 - 总结与展望：总结工作成果、分析不足、展望未来改进方向
- 论文质量要求：
 - 论文字数不少于15000字
 - 论文结构完整，逻辑清晰
 - 图表清晰，格式规范
 - 参考文献不少于20篇，其中外文文献不少于5篇
 - 符合学校毕业论文格式要求
- 技术深度要求：
 - 深入分析多智能体协作机制的设计原理
 - 详细阐述强化学习优化策略的实现方法
 - 分析系统性能优化策略
 - 对比分析不同技术方案的优缺点

二、课题条件及进度安排

2.1 课题条件

2.1.1 硬件条件

- 开发环境：

- 个人计算机：CPU 4核以上，内存16GB以上，硬盘500GB以上
- GPU（可选）：支持CUDA的NVIDIA显卡，用于模型训练和推理加速
- 服务器环境：
 - 云服务器：2核4G起步（用于测试环境）
 - 数据库服务器：MySQL 8.0，Redis 7.0

2.1.2 软件条件

- 开发工具：
 - Python 3.9+ 开发环境
 - Node.js 16+ 开发环境
 - MySQL 8.0 数据库
 - Redis 7.0 缓存数据库
 - Docker & Docker Compose（容器化部署）
 - Git 版本控制工具
 - IDE：VS Code / PyCharm
- 技术框架与库：
 - 后端：FastAPI、SQLAlchemy、LangChain、Transformers、PyTorch
 - 前端：Vue.js 3、Ant Design Vue、Uni-app / Taro
 - AI模型：Qwen2.5-7B-Instruct / ChatGLM、PaddleOCR
 - 其他：Celery、JWT、Pydantic、Alembic
- 数据资源：
 - 公开医疗数据集（Kaggle、MIMIC-III等）
 - 中文医学知识图谱（CMeKG）
 - 临床指南数据库

2.1.3 参考资料

- 学术论文：
 - MDAgents框架相关论文（MIT & Google）
 - DoctorAgent-RL框架相关论文
 - 多智能体系统、强化学习在医疗领域应用的相关文献
- 技术文档：
 - LangChain官方文档
 - FastAPI官方文档
 - Vue.js官方文档
 - 相关开源项目代码
- 设计文档：
 - 项目设计方案文档
 - 代码框架设计文档
 - API接口设计文档

2.1.4 指导教师支持

- 指导教师提供技术指导和方案审查
- 定期进行进度检查和问题讨论
- 协助解决技术难点和资源获取

2.2 进度安排

第一阶段：需求分析与系统设计（第1-2周）

时间安排：2周

主要任务：

- 深入调研多智能体系统在医疗领域的应用现状
- 分析系统功能需求和非功能需求
- 完成系统总体架构设计
- 完成数据库设计（ER图、表结构设计）
- 完成API接口设计
- 完成多智能体协作机制设计
- 编写需求分析文档和系统设计文档

交付物：

- 需求分析文档
- 系统设计文档（包含架构图、数据库设计、接口设计）
- 技术选型说明文档

检查点：

- 需求分析评审通过
- 系统设计方案评审通过

第二阶段：基础框架搭建（第3-5周）

时间安排：3周

主要任务：

- 搭建项目开发环境（Python、Node.js、数据库等）
- 初始化项目结构，配置开发工具
- 实现数据库模型（SQLAlchemy ORM）
- 实现用户认证与权限管理系统（JWT、RBAC）
- 实现基础API接口框架
- 配置Docker开发环境
- 实现日志系统、异常处理机制
- 编写单元测试框架

交付物：

- 后端基础框架代码
- 数据库初始化脚本
- 基础API接口（用户认证、权限管理）
- 单元测试代码

检查点：

- 基础框架功能测试通过
 - 代码审查通过
-

第三阶段：核心功能开发（第6-10周）

时间安排：5周

主要任务：

3.1 患者端功能开发（第6-7周）：

- 实现咨询对话功能（WebSocket实时通信）
- 实现报告上传功能（文件上传、OCR识别）
- 实现用药提醒管理功能（CRUD、定时任务）
- 实现个人中心功能（历史记录查询、个人信息管理）
- 小程序界面开发与优化

3.2 医生端功能开发（第8周）：

- 实现患者管理界面（患者列表、会话详情）
- 实现智能辅助诊断界面（病历展示、编辑、可视化）
- 实现知识库查询功能
- 实现病历归档功能
- Web管理后台界面开发与优化

3.3 后端服务开发（第9-10周）：

- 实现会话管理服务
- 实现病历管理服务
- 实现报告处理服务（OCR、数据提取）
- 实现提醒任务调度服务（Celery）
- 实现知识库查询服务

交付物：

- 患者端小程序代码
- 医生端Web管理后台代码
- 后端核心服务代码
- 功能测试用例

检查点：

- 患者端功能测试通过
 - 医生端功能测试通过
 - 前后端联调测试通过
-

第四阶段：多智能体系统开发（第11-15周）

时间安排：5周

主要任务：

4.1 智能体基础框架开发（第11周）：

- 实现智能体基类和接口定义
- 实现智能体消息通信机制
- 实现智能体调度器（Orchestrator）

- 实现智能体团队构建器（Team Builder）

4.2 调节器代理开发（第11-12周）：

- 实现医学复杂度评估模块
- 实现路由决策模块
- 实现协作模式选择机制

4.3 PCC/MDT/ICT模式开发（第12-14周）：

- 实现PCC模式：单个智能体，CoT推理，自一致性验证
- 实现MDT模式：多智能体迭代讨论，共识达成机制
- 实现ICT模式：分阶段处理，交叉审查，综合报告生成
- 实现专科智能体（CardioMind）：病历结构化、诊断建议生成

4.4 患者模拟智能体开发（第14周）：

- 实现患者模拟智能体
- 实现动态症状释放策略
- 实现病理一致性保证机制

4.5 病历结构化引擎开发（第14-15周）：

- 实现NLP信息抽取模块（NER、关系抽取）
- 实现病历模板匹配与填充
- 实现数据验证与补全机制

交付物：

- 多智能体系统核心代码
- 智能体协作测试用例
- 病历结构化测试用例

检查点：

- 智能体协作功能测试通过
- 病历结构化准确率测试通过

第五阶段：智能体优化与评估系统开发（第16-18周）

时间安排：3周

主要任务：

5.1 多维度奖励评估系统开发（第16周）：

- 实现诊断准确性奖励计算
- 实现信息获取效率奖励计算
- 实现协作效率奖励计算
- 实现遵循规范奖励计算
- 实现总奖励综合计算

5.2 强化学习训练系统开发（第17周）：

- 实现监督微调（SFT）训练流程
- 实现强化学习（RL）训练流程（GRPO算法）
- 实现动态轮数预算训练策略
- 实现训练数据管理
- 实现模型版本管理

5.3 性能监控与评估系统开发（第18周）：

- 实现智能体性能指标统计
- 实现训练过程监控
- 实现评估报告生成
- 实现A/B测试框架

交付物：

- 评估与优化系统代码
- 训练脚本和配置文件
- 性能评估报告

检查点：

- 奖励评估系统功能测试通过
 - 训练流程测试通过
 - 性能指标达到预期目标
-

第六阶段：系统集成与优化（第19-20周）

时间安排：2周

主要任务：

- 完成前后端系统集成
- 完成多智能体系统与业务系统集成
- 进行系统性能优化（缓存、数据库优化、API优化）
- 进行系统安全加固（数据加密、SQL注入防护、XSS防护）
- 修复系统bug，完善错误处理
- 编写系统集成测试用例并执行
- 进行压力测试和性能测试

交付物：

- 完整系统代码
- 集成测试报告
- 性能测试报告
- Bug修复记录

检查点：

- 系统集成测试通过
 - 性能指标达到要求
 - 安全测试通过
-

第七阶段：测试与文档编写（第21-22周）

时间安排：2周

主要任务：

- 编写完整的单元测试（覆盖率≥70%）
- 编写集成测试用例并执行
- 编写系统测试用例并执行
- 编写用户验收测试用例
- 编写系统设计文档
- 编写API接口文档

- 编写用户使用手册
- 编写部署文档
- 编写毕业论文初稿

交付物：

- 测试用例和测试报告
- 系统设计文档
- API接口文档
- 用户使用手册
- 部署文档
- 毕业论文初稿

检查点：

- 测试覆盖率达标
 - 文档完整性检查通过
 - 毕业论文初稿审查通过
-

第八阶段：系统部署与演示准备（第23周）

时间安排：1周

主要任务：

- 配置生产环境服务器
- 部署系统到生产环境
- 配置域名和SSL证书
- 录制系统演示视频
- 准备答辩PPT
- 准备答辩材料

交付物：

- 部署的系统
- 演示视频
- 答辩PPT
- 答辩材料

检查点：

- 系统部署成功
 - 演示视频质量检查通过
 - 答辩材料准备完成
-

第九阶段：论文修改与答辩准备（第24周）

时间安排：1周

主要任务：

- 根据指导教师意见修改论文
- 完善论文图表和格式
- 进行论文查重和修改
- 准备论文答辩
- 进行答辩预演

- 交付物：
- 最终版毕业论文
- 答辩PPT（最终版）
- 答辩准备材料
- 检查点：
- 论文格式符合要求
- 论文查重通过
- 答辩准备完成

2.3 关键里程碑

里程碑	时间节点	主要交付物	检查标准
M1：需求分析与设计完成	第2周末	需求分析文档、系统设计文档	文档评审通过
M2：基础框架搭建完成	第5周末	后端基础框架、数据库模型	功能测试通过
M3：核心功能开发完成	第10周末	患者端、医生端、后端服务	前后端联调通过
M4：多智能体系统开发完成	第15周末	多智能体系统核心代码	智能体协作测试通过
M5：优化系统开发完成	第18周末	评估与优化系统	性能指标达标
M6：系统集成完成	第20周末	完整系统	集成测试通过
M7：测试与文档完成	第22周末	测试报告、技术文档	文档完整性检查通过
M8：系统部署完成	第23周末	部署的系统、演示视频	系统正常运行
M9：论文完成	第24周末	最终版毕业论文	论文查重通过、格式符合要求

2.4 风险控制

2.4.1 技术风险

风险1：AI模型准确性不足

- 应对措施：使用高质量医学知识库，结合医生审核机制，持续优化模型
- 预案：准备多个模型方案，进行对比测试，选择最优方案

风险2：多智能体协作机制复杂，实现难度大

- 应对措施：参考MDAgents框架，分阶段实现，先实现基础功能，再逐步完善
- 预案：简化协作机制，先实现核心功能，后续优化

风险3：系统性能不达标

- 应对措施：提前进行性能测试，优化关键路径，使用缓存机制
- 预案：降低并发要求，优化算法，使用更强大的服务器

2.4.2 进度风险

风险1：开发进度滞后

- 应对措施：制定详细进度计划，每周检查进度，及时调整计划
- 预案：压缩非核心功能，优先保证核心功能完成

风险2：测试时间不足

- **应对措施**：开发过程中同步编写测试用例，边开发边测试
- **预案**：延长测试时间，或简化测试用例

2.4.3 资源风险

风险1：缺乏真实医疗数据

- **应对措施**：使用公开数据集（MIMIC-III、Kaggle等），构造符合逻辑的模拟数据
- **预案**：使用合成数据生成技术，或与医疗机构合作获取数据

风险2：计算资源不足

- **应对措施**：使用云服务器，合理规划资源使用
- **预案**：使用轻量级模型，或使用模型API服务

2.5 质量保证措施

- **代码质量**：
 - 使用代码审查工具（Black、Flake8、MyPy）
 - 编写单元测试，保证测试覆盖率≥70%
 - 遵循编程规范，使用类型提示
- **文档质量**：
 - 及时更新文档，保证文档与代码同步
 - 文档格式规范，内容完整
 - 进行文档评审
- **测试质量**：
 - 编写完整的测试用例
 - 进行多轮测试（单元测试、集成测试、系统测试）
 - 进行性能测试和安全测试
- **进度控制**：
 - 每周进行进度检查
 - 及时识别和解决风险
 - 根据实际情况调整计划

三、预期成果

3.1 系统成果

- **完整的系统**：
 - 患者端小程序（支持健康咨询、报告上传、用药提醒等功能）
 - 医生端Web管理后台（支持患者管理、辅助诊断、知识库查询等功能）
 - 后端服务系统（支持多智能体协同、病历结构化、智能体优化等功能）
- **技术文档**：
 - 系统设计文档
 - API接口文档

- 用户使用手册
- 部署文档
- 测试报告：
 - 单元测试报告
 - 集成测试报告
 - 系统测试报告
 - 性能测试报告

3.2 论文成果

- 毕业论文：不少于15000字，结构完整，内容充实
- 答辩PPT：清晰展示系统设计和实现成果
- 演示视频：展示系统主要功能

3.3 技术成果

- 多智能体协同系统：实现自适应多智能体协作机制
- 智能体优化系统：实现多维度奖励评估和强化学习优化
- 病历结构化引擎：实现从非结构化到结构化的自动转化

指导教师签名：_____ 日期：_____

学生签名：_____ 日期：_____